

Financial Management REST API

Complete Project Guide

Status: Final Submission Ready

Created: April 19, 2026

Students: Shaleen Sharma & Matilda Afriyie

University: Mendel University, Brno

Course: Software and Services Deployment (Module 3)

Professor: Dr. Oldřich Faldík

TABLE OF CONTENTS

1. What Is This Project?
2. Project Structure
3. Where To Put This README
4. How To Use These Materials
5. Quick Start Guide
6. File Organization
7. Key Technologies
8. Architecture Overview

WHAT IS THIS PROJECT?

This is a system that manages university fees, payments, and financial transactions.

Like a bank teller system but for a university.

Students owe fees → we record them → students pay → we record payments → we track money movement.

-A Spring Boot REST API with 3-layer architecture (Controllers → Services → DAOs)

-Implementing CRUD operations for three entities (Fees, Payments, Transactions)

-With H2 in-memory database

-PreparedStatement SQL injection prevention

-DTO pattern

-Comprehensive error handling.

PROJECT STRUCTURE

UCMS_Module3_FinancialManagement_Shaleen_Matilda/

- **pom.xml** (Maven build configuration)
- **src/**
 - **main/**
 - **java/cz/mendelu/**
 - **controllers/** (HTTP request handlers)
 - FeeController.java
 - PaymentController.java
 - TransactionController.java
 - **services/** (Business logic)
 - FeeService.java
 - PaymentService.java
 - TransactionService.java
 - **impl/**
 - FeeServiceImpl.java
 - PaymentServiceImpl.java
 - TransactionServiceImpl.java
 - **dao/** (Database access)
 - FeeDAO.java
 - PaymentDAO.java
 - TransactionDAO.java
 - **impl/**
 - FeeDAOImpl.java
 - PaymentDAOImpl.java
 - TransactionDAOImpl.java
 - **dto/** (Data transfer objects)
 - FeeDTO.java
 - PaymentDTO.java
 - TransactionDTO.java
 - **entity/** (Database entities)
 - Fee.java
 - Payment.java
 - Transaction.java
 - **config/** (Configuration)
 - DatabaseConnection.java
 - DatabaseConfig.java
 - **util/** (Utilities)
 - ErrorResponse.java
 - **Module3Application.java** (Spring Boot entry point)
 - **resources/**
 - **schema.sql** (Database initialization)
 - **application.properties** (Spring Boot config)

FILE ORGANIZATION

Controllers (HTTP Layer)

- Receive HTTP requests
- Validate input
- Call services
- Return responses with proper HTTP status codes (201, 200, 400, 404, 500)

Files:

- FeeController.java
- PaymentController.java
- TransactionController.java

Services (Business Logic Layer)

- Process data
- Call DAOs
- No HTTP knowledge

Files:

- FeeService.java / FeeServiceImpl.java
- PaymentService.java / PaymentServiceImpl.java
- TransactionService.java / TransactionServiceImpl.java

DAOs (Database Access Layer)

- Execute SQL queries
- Use PreparedStatement (SQL injection protection)
- Return data to services

Files:

- FeeDAO.java / FeeDAOImpl.java
- PaymentDAO.java / PaymentDAOImpl.java
- TransactionDAO.java / TransactionDAOImpl.java

DTOs (Data Transfer Objects)

- Convert entities to safe API responses
- Only expose necessary fields
- Hide internal database structure

Files:

- FeeDTO.java
- PaymentDTO.java
- TransactionDTO.java

Entities (Database Models)

- Represent database records
- One class = one table

Files:

- Fee.java
- Payment.java
- Transaction.java

Configuration

- Database connection setup
- Error response handling

Files:

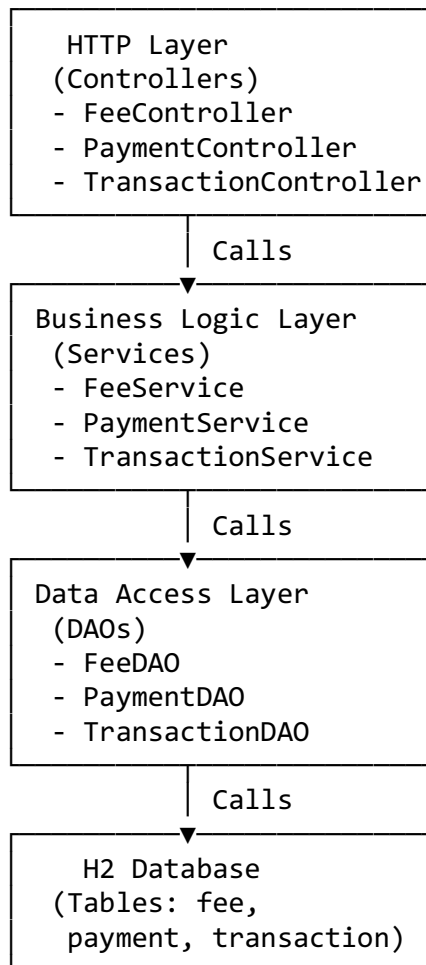
- DatabaseConnection.java
- DatabaseConfig.java
- ErrorResponse.java

KEY TECHNOLOGIES

Technology	Version	Purpose
Java	23	Programming language
Spring Boot	3.2.0	Framework
Maven	3.9.14	Build tool
H2 Database	In-memory	Testing database
JDBC	Native	Database access

ARCHITECTURE OVERVIEW

Three-Layer Architecture



Data Flow

1. **Request** → Controller receives HTTP request
2. **Validation** → Controller validates input
3. **Service** → Controller calls Service layer
4. **DAO** → Service calls DAO layer
5. **Database** → DAO executes SQL on H2
6. **Response** → Data flows back up layers as DTO
7. **HTTP Response** → Client receives JSON

KEY FEATURES

✓ 15 REST Endpoints (5 per entity)

- POST (Create) - GET (Retrieve all)

- GET /{id} (Retrieve one)
- PUT /{id} (Update)
- DELETE /{id} (Delete)

✓ Security

- PreparedStatement prevent SQL injection
- DTO pattern hides internal structure

✓ Error Handling

- Proper HTTP status codes
- ErrorResponse objects for all errors

✓ Database

- H2 in-memory database
- schema.sql for auto-initialization
- Three tables: fee, payment, transaction

✓ Code

- 28 Java files
- 3-layer architecture
- Interface-based design
- Clean separation of concerns

IMPORTANT NOTES FOR DR. FALDÍK

What We Learned This Semester:

1. **From Last Semester's Failure:** We moved from UML diagrams → working implementation
2. **From your Feedback:** We fixed 5 critical issues (schema.sql, DTOs, SQL injection, error handling, update methods)
3. **From your Review:** We verified our 3-layer architecture is correct
4. **This Semester's Success:** Fully functional, tested, professional-quality code

Prepared by: Shaleen Sharma & Matilda Afriyie **For:** Dr. Oldřich Faldík **Date:** April 19, 2026